

Les 5 familles d'architectures RAG

Du RAG naïf à l'agent qui planifie, vérifie et corrige ses sources — avec boîte à outils, reranking, évaluation et guide de décision

Module 0 — Le socle : c'est quoi, un RAG ?
Annexe A — Boîte à outils (chunking, embedding, top-k...)
Module 1 — Hybrid RAG + BM25/RRF chiffré + reranking
Module 2 — Graph RAG
Module 3 — Agentic RAG
Module 4 — Corrective RAG (CRAG)
Module 5 — Multimodal RAG
Module 6 — Évaluer son RAG (golden set, métriques)
Guide de décision — quelle famille choisir
Erreurs classiques
Architecture de référence (production)
Corpus fil rouge + TP niveaux 1 à 4
Glossaire & Références

v2 — réponses des checkpoints et du quiz regroupées en fin de document pour un vrai auto-test

Objectif : comprendre le principe indexer → récupérer → générer, et pourquoi le RAG naïf a des angles morts.

RAG = Retrieval-Augmented Generation (Génération Augmentée par Recherche).



Un LLM seul = étudiant sans documents (récite, hallucine). Le RAG = examen livre ouvert.

⚠ Le RAG « naïf » a 4 faiblesses : il rate les mots-clés exacts, ignore les relations entre documents, fait confiance aveuglément à ce qu'il retrouve, et ne sait pas lire un graphique. **Les 5 architectures suivantes sont 5 remèdes à ces faiblesses.** Le module 6 ajoute un 6e remède : mesurer si le remède fonctionne vraiment.

Objectif : donner un sens concret à chunking, embedding, similarité cosinus, base vectorielle et top-k avant de les voir en action.

Terme	Définition concrète
Chunking	Découpage d'un document en fragments (chunks) de taille fixe (ex. 300-500 tokens) avec un overlap (chevauchement, ex. 50 tokens) entre fragments consécutifs pour ne pas couper une idée à la frontière. Trop petit → perd le contexte ; trop grand → dilue le signal utile dans le top-k.
Embedding	Fonction qui transforme un texte en vecteur de nombres (ex. 768 dimensions) tel que deux textes de sens proche ont des vecteurs proches dans l'espace. C'est ce qui permet de chercher par « sens » plutôt que par mot exact.
Similarité cosinus	Mesure de l'angle entre deux vecteurs (1 = identiques en direction, 0 = sans rapport). Sert à classer les documents indexés par proximité de sens avec la question.
Base vectorielle	Index spécialisé dans la recherche du plus proche voisin sur des millions de vecteurs (FAISS, Chroma, Qdrant, pgvector...). Sans elle, comparer la question à chaque document un par un serait trop lent.
Top-k	Nombre de fragments les mieux classés effectivement transmis au LLM comme contexte. k trop bas → le LLM manque d'info ; k trop haut → le contexte est noyé et le LLM peut se perdre ou halluciner.

💡 Ces cinq briques suffisent à construire un RAG naïf. Chaque module qui suit ajoute une brique par-dessus pour combler une faiblesse précise.

01

Hybrid RAG

« Meaning + exact matches » — le sens ET la lettre

Objectif : savoir pourquoi et comment combiner recherche sémantique et recherche lexicale, puis pourquoi ajouter un reranker derrière.



Principe

On traduit question + documents en vecteurs pour capturer le **sens** (« connexion lente » ≈ « réseau instable »), tout en gardant une recherche statistique de mots exacts (**BM25**) pour les références, codes, noms propres (ERR-4042, article L.1234-5) que les vecteurs noient. Les deux classements sont ensuite fusionnés, puis on garde les k meilleurs fragments.

BM25 + Rank Fusion (RRF), chiffré

La fusion la plus utilisée est le **Reciprocal Rank Fusion** : chaque document reçoit un score $1/(k+\text{rang})$ dans chaque liste (k=60 par convention), puis on additionne.

```
Liste vecteurs (par rang) : d1, d2, d3
Liste BM25 (par rang) : d3, d1, d4

score(d1) = 1/(60+1) + 1/(60+2) = 0,01639 + 0,01613 = 0,03252
score(d3) = 1/(60+3) + 1/(60+1) = 0,01587 + 0,01639 = 0,03226
score(d2) = 1/(60+2) = 0,01613
score(d4) = 1/(60+3) = 0,01587

Ordre fusionné : d1 > d3 > d2 > d4
```

Un document présent dans **les deux** listes (d1, d3) bat systématiquement un document présent dans une seule (d2, d4), même s'il n'est pas 1er dans chacune.

Le pattern retrieve → rerank

En production, l'hybrid retrieval seul reste **rapide mais approximatif** (optimisé pour le rappel : ne rien louper). On ajoute un **reranker cross-encoder** : un modèle qui lit la paire (question, document) ensemble — plus lent, mais bien plus précis — et réordonne les candidats avant la coupe top-k finale.



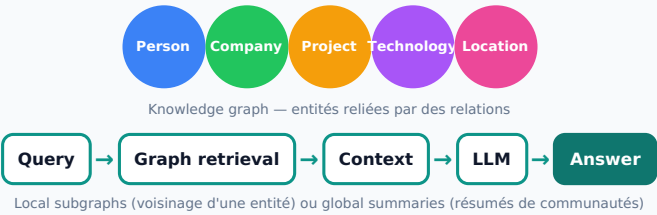
Quand ?	Choix par défaut, surtout corpus techniques/jargonneux avec identifiants
Limite	Ne comprend toujours pas les relations entre documents ; sans reranker, precision plus faible
À retenir	Deux classements valent mieux qu'un ; un reranker derrière vaut mieux que deux

✔ **Checkpoint 1** : un utilisateur tape « erreur POL-2025-114 ». Quelle branche le sauve, et pourquoi un reranker seul (sans BM25) ne suffirait pas forcément ici ?

02 Graph RAG

« Answers in relationships » — la réponse est dans les liens

Objectif : reconnaître une question relationnelle multi-sauts et savoir pourquoi le chunk seul ne peut pas y répondre.



Principe

La connaissance n'est plus une pile de chunks isolés : ce sont des **entités reliées par des relations** (Marie → travaille pour → Acme → participe à → Projet H2...). Les **sous-graphes locaux** permettent un raisonnement multi-sauts ; les **résumés globaux** répondent aux questions macro.

Quand ?	Questions relationnelles (qui travaille avec qui, sur quoi, où), traçabilité, synthèses globales
Limite	Construction coûteuse (extraction entités/rerelations) et maintenance du graphe
À retenir	Souvent, aucun chunk ne contient la réponse : elle existe seulement entre les chunks

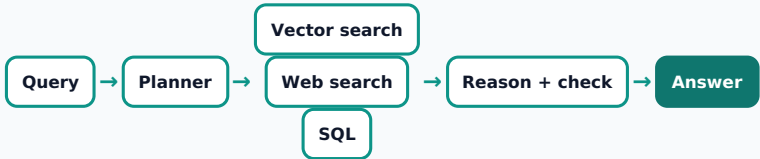
✓ **Checkpoint 2** : « Quelles entreprises partenaires utilisent la techno X dans des projets communs ? » — pourquoi le Hybrid RAG du module 1 galère ici ?

03

Agentic RAG

« Plan. Retrieve. Verify. » — la récupération devient une stratégie

Objectif : comprendre le rôle du planner, de la boucle de vérification, et pourquoi un budget d'itérations est non négociable.



↳ Retry within budget : « Reason + check » peut relancer le Planner, dans la limite d'un budget d'itérations

Principe

Un **agent** décide quoi chercher, où, dans quel ordre : base interne (vecteurs), web, ou base de données (SQL). Après chaque récupération, il **raisonne et vérifie** ; si c'est insuffisant, il **relance** — mais seulement dans la limite d'un **budget** (nombre d'appels, temps), sinon il répond en l'état.

Quand ?	Questions multi-sources (ventes en SQL + tendances marché en web + note interne en vecteurs)
Limites	Latence et coût élevés, débogage difficile → budget obligatoire sous peine de boucles infinies
À retenir	Ce n'est plus un tuyau, c'est un pilote : planifier, récupérer, vérifier, et savoir s'arrêter

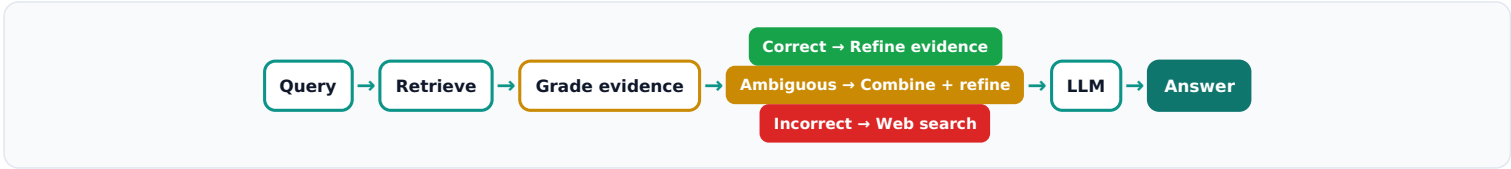
✔ **Checkpoint 3** : qu'empêche la boucle « Retry within budget » de tourner indéfiniment ?

04

Corrective RAG (Crag)

« Grade before you trust » — note avant de croire

Objectif : comprendre pourquoi noter la récupération avant génération réduit les hallucinations, et ce que fait chaque verdict.



Principe

Un **évaluateur** (scoreur) juge la qualité des documents retrouvés **avant** de les donner au LLM : la récupération n'est plus une vérité, c'est une **hypothèse notée**. Correct → on raffine ; ambigu → on combine avec un complément ; incorrect → on part chercher sur le web avant de combiner.

Quand ?	Corpus bruités/hétérogènes/mouvants, besoin de robustesse et de filet de sécurité
Limites	Un composant de plus à calibrer (le gradeur), latence ajoutée
À retenir	Trois verdicts, trois destins : raffiner, combiner, ou sortir chercher ailleurs

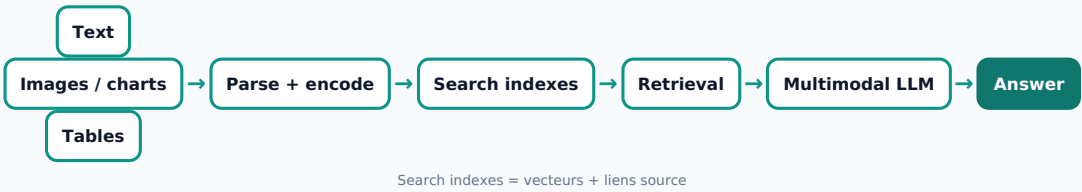
✓ **Checkpoint 4** : documents retrouvés hors-sujet (verdict Incorrect) : que fait Crag ?

05

Multimodal RAG

« Go beyond text » — indexe ce que tu lis, y compris ce que tu vois

Objectif : savoir quand l'information vit dans une figure/un tableau plutôt que dans le texte, et comment l'indexer.



Principe

L'information vitale dort souvent dans une **courbe, un schéma, un tableau**. Deux voies d'encodage : **embeddings multimodaux** (texte et image dans un même espace vectoriel) ou **captioning** (un modèle décrit l'image, puis index texte). Les **liens source** permettent de citer/afficher l'extrait original.

Quand ?	PDF techniques/financiers, slides, rapports où figures et tables portent le sens
Limites	Qualité des embeddings multimodaux, alignement des modalités, coût
À retenir	Un chunk n'est pas forcément un paragraphe : ça peut être une figure + légende + tableau

06

Évaluer son RAG

Sans mesure, « comparer » n'est qu'une impression

Objectif : construire un golden set et savoir lire les 4 métriques standard avant de déclarer qu'une architecture est « meilleure ».

Le golden set

Un jeu de référence de **questions + réponses attendues + sources attendues**, construit à la main sur ton propre corpus (10 à 30 paires suffisent pour démarrer). C'est la seule façon de comparer deux architectures sur un même étalon plutôt qu'au feeling.

Les 4 métriques

Métrique	Ce qu'elle mesure
Context precision	Parmi les fragments récupérés, quelle proportion est réellement pertinente ? (mesure le bruit)
Context recall	Parmi les fragments réellement pertinents qui existent dans le corpus, quelle proportion a été récupérée ? (mesure les oublis)
Faithfulness (fidélité)	La réponse générée s'appuie-t-elle uniquement sur le contexte fourni, ou invente-t-elle des éléments absents ? (mesure l'hallucination)
Answer relevance	La réponse répond-elle effectivement à la question posée, même si elle est fidèle au contexte ? (mesure le hors-sujet)

🔗 Des frameworks comme **RAGAS** calculent ces 4 métriques automatiquement à partir du golden set, du contexte récupéré et de la réponse générée — pas besoin de tout noter à la main au-delà du prototype.

⚠ Un bon context recall avec une mauvaise faithfulness signale un problème de **génération** (le LLM ignore le contexte) ; l'inverse signale un problème de **récupération** (le contexte est incomplet). Le diagnostic dépend de quelle métrique décroche.



Guide de décision

Quelle famille choisir, dans quel ordre se poser les questions

Arbre de décision (à parcourir dans l'ordre, premier « oui » gagne)

- 1 Le corpus contient des **références/codes exacts** à retrouver mot pour mot ? → **Hybrid RAG** (+ reranker)
- 2 La question porte sur des **relations entre entités** (qui, avec qui, sur quel projet) ? → **Graph RAG**
- 3 La réponse exige de **croiser plusieurs sources hétérogènes** (SQL + web + interne) ? → **Agentic RAG**
- 4 Le corpus est **bruité, mouvant ou incomplet**, avec risque de mauvais retrieval ? → **Corrective RAG**
- 5 Le sens est porté par des **figures, tableaux, graphiques** ? → **Multimodal RAG**
- 6 Aucun cas ci-dessus ne domine ? → **Hybrid RAG** par défaut, puis instrumenter avec le module 6 avant d'ajouter de la complexité

Coût / latence / complexité / fiabilité

Architecture	Problème ciblé	Complexité	Latence	Fiabilité
Hybrid RAG	Rappel faible sur termes exacts	Faible	Faible	Moyenne
Graph RAG	Questions relationnelles multi-hop	Élevée	Moyenne	Moyenne-haute
Agentic RAG	Sources hétérogènes, raisonnement	Élevée	Haute	Variable (dépend du planner)
Corrective RAG	Hallucination sur contexte pauvre	Moyenne	Moyenne	Haute
Multimodal RAG	Contenu non-textuel	Élevée	Moyenne-haute	Moyenne-haute

🔗 Ces patterns se combinent : rien n'empêche un Graph RAG noté par un gradeur CRAG, ou un Agentic RAG dont l'outil « vector search » est lui-même un Hybrid RAG + reranker.



Erreurs classiques

Ce qui casse un RAG en silence

- **Chunks trop gros** : chaque fragment mélange plusieurs idées, le vecteur résultant est flou et le retrieval devient imprécis.
- **Overlap nul** : une idée coupée pile à la frontière entre deux chunks devient invisible pour les deux.
- **Top-k trop élevé** : le contexte noie le signal utile dans du bruit, le LLM peut se perdre ou halluciner sur un détail périphérique.
- **Pas de reranking** : bon rappel, mauvaise precision — le LLM reçoit des candidats pertinents mais mal classés.
- **Absence de citations** : impossible de vérifier ou tracer d'où vient une affirmation de la réponse.
- **Gradeur CRAG non calibré** : trop strict, il déclenche des web search inutiles ; trop laxiste, il laisse passer du mauvais contexte.
- **Agent sans budget** : boucles de retry coûteuses, voire infinies, sans garantie de réponse dans un temps raisonnable.



Architecture de référence

Ce à quoi ressemble un système RAG de production



Les modules Graph, Agentic et Multimodal ne remplacent pas ce socle : ils l'enrichissent ou l'orchestrent (un agent peut appeler ce pipeline comme un outil parmi d'autres ; un Graph RAG peut l'utiliser comme récupérateur de secours sur les entités non couvertes par le graphe).

Corpus fil rouge (fictif, ~20 pages)

Réutilisé dans les 4 TP pour que les résultats soient réellement comparables d'une architecture à l'autre :

Composant	Contenu
Notes de support	Tickets avec identifiants exacts (ex. ERR-4042, POL-2025-114) — pour tester la branche BM25
Contrat	Articles numérotés et clauses croisées entre entreprises/personnes/projets — pour tester le Graph RAG
Rapport avec courbes	Quelques pages avec un graphique de ventes trimestrielles et un tableau — pour tester le Multimodal RAG

TP progressifs

- Niveau 1** — RAG naïf sur le corpus fil rouge (chunks + embeddings + top-k), puis ajoute BM25 + fusion RRF, puis un reranker. Compare sur 5 questions dont 2 avec références exactes.
- Niveau 2** — construis un mini-graphe (entités/rerelations) à partir du contrat ; compare hybrid vs graph sur 3 questions relationnelles.
- Niveau 3** — ajoute un gradeur LLM devant ta récupération (CRAG) ; teste sur des questions sans réponse dans le corpus et observe le repli web.
- Niveau 4** — construis un golden set de 10 paires question/réponse sur le corpus fil rouge ; calcule les 4 métriques du module 6 (à la main ou avec RAGAS) pour chacune des architectures testées aux niveaux 1 à 3.

#	Famille	Devise	Super-pouvoir	Cas type	Vigilance
01	Hybrid	Sens + exact	Rappel maximal	Jargon, références	Réglage fusion + reranker
02	Graph	Réponses dans les liens	Multi-sauts + synthèses globales	« Qui avec qui, sur quoi ? »	Coût de construction
03	Agentic	Plan. Retrieve. Verify.	Multi-sources adaptatif	Questions complexes	Budget, latence
04	Corrective	Note avant de croire	Robustesse, filet web	Corpus bruités	Calibration du gradeur
05	Multimodal	Au-delà du texte	Figures & tables interrogables	PDF riches	Embeddings multimodaux
06	Évaluation	Mesurer avant d'affirmer	Comparaison objective	Choisir/arbitrer entre familles	Golden set représentatif

🔮 **Ces familles se combinent.** Un système production sérieux = hybride + reranker (01) + gradeur (04) + touches multimodales (05), parfois orchestré par un agent (03), et toujours mesuré (06).

Quiz d'auto-évaluation

- Question « contrat DC-2024-091 » : quelle branche de l'Hybrid RAG est décisive ?
- « Résume les grands thèmes de tout le corpus » : quel mode du Graph RAG ?
- Dans l'Agentic RAG, quel mécanisme borne les relances ?
- Dans CRAG, verdict « Incorrect » → quelle action ?
- Répondre sur un graphique d'un PDF : quelle famille ?
- Quel composant de la bande 01 fusionne les deux classements ?
- Un reranker cross-encoder est-il plus rapide ou plus lent qu'un bi-encoder, et pourquoi l'utiliser quand même ?
- Bon context recall mais mauvaise faithfulness : le problème vient de la récupération ou de la génération ?
- Dans l'arbre de décision, quel est le choix par défaut si aucun critère ne domine ?

Réponses en fin de document, avec celles des checkpoints 1 à 4.



Glossaire

15 termes pour ne plus décrocher

RAG

Retrieval-Augmented Generation — génération augmentée par une étape de recherche documentaire.

Chunk

Fragment de document obtenu par découpage (chunking), unité de base indexée et récupérée.

Embedding

Représentation vectorielle d'un texte capturant son sens.

Similarité cosinus

Mesure de proximité entre deux vecteurs, utilisée pour classer les résultats du vector search.

Base vectorielle

Index optimisé pour la recherche de plus proches voisins parmi des embeddings (FAISS, Chroma, Qdrant...).

BM25

Algorithme de recherche lexicale par pertinence statistique des mots exacts.

RRF (Reciprocal Rank Fusion)

Méthode de fusion de plusieurs classements en un seul, basée sur l'inverse du rang.

Reranker / cross-encoder

Modèle qui réordonne un ensemble restreint de candidats en évaluant chaque paire (question, document) conjointement.

Top-k

Nombre de fragments finalement transmis au LLM comme contexte.

Knowledge graph

Représentation de la connaissance sous forme d'entités et de relations entre elles.

Multi-hop

Question dont la réponse nécessite de traverser plusieurs relations/documents successivement.

Agent

Composant qui planifie ses actions (quels outils appeler, dans quel ordre) plutôt que de suivre un pipeline fixe.

Budget (agentic)

Limite de nombre d'itérations ou de temps imposée à un agent pour garantir une réponse finale.

Fidélité (faithfulness)

Degré auquel une réponse générée reste ancrée dans le contexte fourni, sans invention.

Golden set

Jeu de questions/réponses de référence servant d'étalon pour évaluer un système RAG.



Références

Pour aller plus loin

Papiers fondateurs

- Lewis et al., 2020 — introduction du RAG (retrieval-augmented generation) pour les tâches génératives à forte intensité de connaissances.
- Edge et al., 2024 — GraphRAG (Microsoft), retrieval augmenté par graphe de connaissances et résumés de communautés.
- Yan et al., 2024 — Corrective RAG (CRAG), évaluation de la qualité du contexte récupéré avant génération.

Outils

- **Frameworks RAG** : LlamaIndex, LangChain, Haystack.
- **Bases vectorielles** : FAISS, Chroma, Qdrant, pgvector.
- **Recherche lexicale** : rank_bm25 (Python).
- **Évaluation** : RAGAS, TruLens, DeepEval.



Réponses

Checkpoints 1-4 et quiz — à consulter après avoir répondu

Checkpoints

Checkpoint 1 — BM25 sauve la mise car la référence exacte est rarement proche vectoriellement d'autre chose qu'elle-même. Un reranker seul ne suffit pas s'il ne voit jamais le bon document dans sa liste de candidats initiale : il réordonne, il ne récupère pas.

Checkpoint 2 — le Hybrid RAG cherche des chunks pertinents individuellement ; ici, la réponse dépend de relations entre plusieurs entités (entreprise ↔ techno ↔ projet) qu'aucun chunk isolé ne relie explicitement.

Checkpoint 3 — le budget (nombre d'appels ou de temps maximum), qui force l'agent à répondre en l'état plutôt que de relancer indéfiniment.

Checkpoint 4 — repli sur web search, puis combinaison avec le contexte existant, au lieu d'halluciner sur des documents hors-sujet.

Quiz

1. BM25 (mots exacts).
2. Global summaries.
3. Le budget (retry within budget).
4. Repli sur Web search puis combinaison.
5. Multimodal RAG.
6. Rank fusion.
7. Plus lent (il évalue chaque paire conjointement) ; on l'utilise quand même car il est bien plus précis, sur un petit nombre de candidats déjà pré-filtrés par une recherche rapide.
8. De la génération : le contexte contient l'information nécessaire (bon recall) mais le LLM ne s'y ancre pas correctement (mauvaise fidélité).
9. Hybrid RAG par défaut, puis instrumenter avec le module 6 avant d'ajouter de la complexité.